

CSI2110 Lab 4 – Data Structures & Algorithms

Fatima Jomaa 300491557

Yui Hei Wong 300485139

Due Saturday July 11, 2026

TA Zhuoran Xu

Description of the implementation and design choices:

HeapSort

```
1  ✓ public class HeapSort {
2
3      // Public method called from main
4  ✓  public void heapSort(StudentRecord[] students) {
5
6      if (students == null || students.length <= 1) {
7          return;
8      }
9
10     int n = students.length;
11
12     //Build max heap
13     buildMaxHeap(students);
14
15
16     //Repeatedly remove maximum element
17     for (int i = n - 1; i > 0; i--) {
18
19         //Move current maximum to the end
20         swap(students, 0, i);
21
22         //Restore heap property
23         heapify(students, i, 0);
24     }
25 }
26
27
28 //Build a max heap
29 ✓ private void buildMaxHeap(StudentRecord[] students) {
30
31     int n = students.length;
32
33     //Start from last non-leaf node
34     for (int i = n / 2 - 1; i >= 0; i--) {
35
36         heapify(students, n, i);
37     }
38 }
39
40
41 //Restore heap property
42 ✓ private void heapify(StudentRecord[] students, int heapSize, int index) {
43
44     int largest = index;
45
46     int leftChild = 2 * index + 1;
47     int rightChild = 2 * index + 2;
48
49
50     // Check if left child is larger
51     if (leftChild < heapSize &&
52         students[leftChild].getGrade() > students[largest].getGrade()) {
53
54         largest = leftChild;
55 }
```

```

56
57
58     // Check if right child is larger
59     if (rightChild < heapSize &&
60         students[rightChild].getGrade() > students[largest].getGrade()) {
61
62         largest = rightChild;
63     }
64
65
66     // If largest is not the parent, swap and continue
67     if (largest != index) {
68
69         swap(students, index, largest);
70
71         heapify(students, heapSize, largest);
72     }
73 }
74
75
76 // Swap two StudentRecord objects
77 ✓ private void swap(StudentRecord[] students, int i, int j) {
78
79     StudentRecord temp = students[i];
80     students[i] = students[j];
81     students[j] = temp;
82 }
83 }

```

This is the HeapSort class, which has been implemented for in-place array sorting based on a binary heap structure. It contains methods without explicit instance variables. heapSort(), buildMaxHeap(), heapify(), and swap() are the core methods to operate this sorting mechanism. heapSort() coordinates the entire process by converting an unsorted array into a max-heap and repeatedly moving the root maximum element to the active boundary of the array. buildMaxHeap() transforms the input array data into a valid heap configuration by invoking heapify on all non-leaf subtrees in a bottom-up sequence. heapify() maintains the max-heap property recursively by evaluating a parent node against its left and right children and propagating downward to correct any value placement anomalies based on student grade values. swap() alters index positions of two discrete elements within the storage array to execute structural replacements.

Main

```

1  ✓ public class Main {
2      public static volatile int preventDCE = 0;
3
4  ✓  public static void main(String[] args) {
5
6      //create a students array with at least 10 test cases for parts A, B & C
7      StudentRecord[] students = {
8          new StudentRecord(1001, "Alice", 82.5, 5),
9          new StudentRecord(1002, "Bob", 91.0, 2),
10         new StudentRecord(1003, "Charlie", 75.5, 7),
11         new StudentRecord(1004, "David", 88.0, 1),
12         new StudentRecord(1005, "Emma", 69.5, 4),
13         new StudentRecord(1006, "Fatima", 95.0, 3),
14         new StudentRecord(1007, "George", 81.0, 6),
15         new StudentRecord(1008, "Hannah", 78.5, 8),
16         new StudentRecord(1009, "Ian", 86.5, 9),
17         new StudentRecord(1010, "Julia", 92.0, 2)
18     };
19
20     //PART A:
21
22     MergeSort sorter = new MergeSort();
23     System.out.println("=====PART A=====");
24     System.out.println("Before sorting:");
25
26     for (StudentRecord student : students) {
27         System.out.println(student);
28     }
29
30     sorter.mergeSort1(students);
31
32     System.out.println("\nAfter sorting using Merge Sort:");
33     for (StudentRecord student : students) {
34         System.out.println(student);
35     }
36
37
38     //PART B:
39     System.out.println("\n\n=====PART B=====");
40     System.out.println("Before sorting:");
41
42     for (StudentRecord student : students) {
43         System.out.println(student);
44     }
45
46     //create object
47     QuickSort sorter2 = new QuickSort();
48
49     sorter2.quickSort(students);
50
51     System.out.println("\nAfter sorting using Quick Sort:");
52     for (StudentRecord student : students) {
53         System.out.println(student);
54     }
55

```

Find definitions
symbols in this f
code.

 Filter syr

✓ class M
 func m
 func g
 func d

```

57         //PART C:
58         //create object
59         HeapSort sorter3 = new HeapSort();
60
61         System.out.println("\n\n=====PART C=====");
62         System.out.println("Before sorting:");
63         for(StudentRecord student : students) {
64             System.out.println(student);
65         }
66
67         sorter3.heapSort(students);
68
69         System.out.println("\n\nAfter sorting using Heap Sort:");
70         for(StudentRecord student : students) {
71             System.out.println(student);
72         }
73
74
75         //PART D: Sorting Algorithm Comparison
76         System.out.println("\n\n=====PART D - SORTING ALGORITHM COMPARISON=====");
77
78         //Test with different dataset sizes: 10, 100, 1000
79         //Test at least three input cases: random order, already sorted, and reverse sorted.
80         int[] dataSizes = {10, 100, 1000};
81         String[] dataTypes = {"Random", "Sorted", "Reverse Sorted"};
82
83         System.out.println("\n\nComparison Table (Execution Time in nanoseconds):\n");
84         System.out.printf("%-15s %-15s %-20s %-20s %-20s %-20s\n",
85             "Dataset Type", "Size", "Merge Sort", "Quick Sort", "Heap Sort", "Simple Sort");
86         System.out.println(new String(new char[110]).replace('\0', '-'));
87
88         SimpleSort simpleSort = new SimpleSort();
89         MergeSort mergeSortComparison = new MergeSort();
90         QuickSort quickSortComparison = new QuickSort();
91         HeapSort heapSortComparison = new HeapSort();
92
93         for (String dataType : dataTypes) {
94             for (int size : dataSizes) {
95                 StudentRecord[] testData = generateTestData(size, dataType);
96
97                 // ===== 1. MERGE SORT BENCHMARK =====
98                 StudentRecord[] data1 = testData.clone();
99                 long startTime = System.nanoTime();
100                 mergeSortComparison.mergeSort1(data1);
101                 long mergeSortTime = System.nanoTime() - startTime;
102                 // Force data flow dependency
103                 if (data1.length > 0 && data1[0] != null) {
104                     preventDCE += data1[0].getStudentNumber() + (int)data1[data1.length - 1].getGrade();
105                 }
106

```

```

107         // ===== 2. QUICK SORT BENCHMARK =====
108         StudentRecord[] data2 = testData.clone();
109         long startTimeQ = System.nanoTime();
110         quickSortComparison.quickSort(data2);
111         long quickSortTime = System.nanoTime() - startTimeQ;
112         // Core fix: Force JVM to execute 500k loop iterations to compute first/last element values
113         if (data2.length > 0 && data2[0] != null) {
114             preventDCE += data2[0].getStudentNumber() + (int)data2[data2.length - 1].getGrade();
115         }
116
117         // ===== 3. HEAP SORT BENCHMARK =====
118         StudentRecord[] data3 = testData.clone();
119         long startTimeH = System.nanoTime();
120         heapSortComparison.heapSort(data3);
121         long heapSortTime = System.nanoTime() - startTimeH;
122         if (data3.length > 0 && data3[0] != null) {
123             preventDCE += data3[0].getStudentNumber() + (int)data3[data3.length - 1].getGrade();
124         }
125
126         // ===== 4. SIMPLE SORT BENCHMARK =====
127         StudentRecord[] data4 = testData.clone();
128         long startTimeS = System.nanoTime();
129         simpleSort.sortByGrade(data4);
130         long simpleSortTime = System.nanoTime() - startTimeS;
131         if (data4.length > 0 && data4[0] != null) {
132             preventDCE += data4[0].getStudentNumber() + (int)data4[data4.length - 1].getGrade();
133         }
134
135         System.out.printf("%-15s %-15d %-20d %-20d %-20d %-20d\n",
136             dataType, size, mergeSortTime, quickSortTime, heapSortTime, simpleSortTime);
137     }
138 }
139 // Output this value at the end of the loop to ensure the variable is not eliminated by JIT
140 System.out.println("[Anti-DCE Verification Hash: " + preventDCE + "]);
141
142 // Testing sorting by different keys
143 System.out.println("\n\n--- Testing Sorting by Different Keys ---");
144
145 // Create fresh test data for sorting by grade
146 StudentRecord[] testDataByGrade = {
147     new StudentRecord(1005, "Emma", 69.5, 4),
148     new StudentRecord(1001, "Alice", 82.5, 5),
149     new StudentRecord(1010, "Julia", 92.0, 2),
150     new StudentRecord(1003, "Charlie", 75.5, 7),
151     new StudentRecord(1006, "Fatima", 95.0, 3)
152 };
153
154 // Create fresh test data for sorting by student number
155 StudentRecord[] testDataByStudentNumber = {
156     new StudentRecord(1005, "Emma", 69.5, 4),
157     new StudentRecord(1001, "Alice", 82.5, 5),
158     new StudentRecord(1010, "Julia", 92.0, 2),
159     new StudentRecord(1003, "Charlie", 75.5, 7),
160     new StudentRecord(1006, "Fatima", 95.0, 3)

```

```

161     };
162
163     System.out.println("\n--- Sort by Grade (ascending) ---");
164     System.out.println("Before sorting by grade:");
165     for (StudentRecord student : testDataByGrade) {
166         System.out.println(student);
167     }
168
169     simpleSort.sortByGrade(testDataByGrade);
170     System.out.println("\nAfter sorting by grade (using Insertion Sort):");
171     for (StudentRecord student : testDataByGrade) {
172         System.out.println(student);
173     }
174
175     System.out.println("\n--- Sort by Student Number (ascending) ---");
176     System.out.println("Before sorting by student number:");
177     for (StudentRecord student : testDataByStudentNumber) {
178         System.out.println(student);
179     }
180
181     simpleSort.sortByStudentNumber(testDataByStudentNumber);
182     System.out.println("\nAfter sorting by student number (using Insertion Sort):");
183     for (StudentRecord student : testDataByStudentNumber) {
184         System.out.println(student);
185     }
186
187     // PART E: String Pattern Matching
188     demonstratePatternMatching();
189 }
190
191 // Helper method to generate test data
192 private static StudentRecord[] generateTestData(int size, String dataType) {
193     StudentRecord[] data = new StudentRecord[size];
194
195     for (int i = 0; i < size; i++) {
196         int studentNumber = 1000 + i;
197         String name = "Student" + i;
198         double grade;
199
200         if ("Random".equals(dataType)) {
201             grade = 50 + Math.random() * 50; // Random grades between 50-100
202         } else if ("Sorted".equals(dataType)) {
203             grade = 50 + (i * 50.0 / size); // Ascending order
204         } else { // Reverse Sorted
205             grade = 100 - (i * 50.0 / size); // Descending order
206         }
207
208         data[i] = new StudentRecord(studentNumber, name, grade, i);
209     }
210
211     return data;
212 }

```

```

213
214 // Helper method for Part E - String Pattern Matching demonstration
215 private static void demonstratePatternMatching() {
216     System.out.println("\n\n=====PART E - STRING PATTERN MATCHING=====");
217
218     PatternMatcher matcher = new PatternMatcher();
219
220     // Create at least 3 TextDocument objects for testing
221     TextDocument doc1 = new TextDocument(1, "Computer Science",
222         "Computer science uses data structures and data algorithms");
223     TextDocument doc2 = new TextDocument(2, "Java Tutorial",
224         "Learn Java programming with patterns and design patterns");
225     TextDocument doc3 = new TextDocument(3, "Pattern Matching",
226         "pattern matching is a pattern recognition technique for patterns in data");
227
228     // Create a document with overlapping patterns for proper testing
229     TextDocument overlapDoc = new TextDocument(5, "Overlap Test", "aaaa bbbb cccc");
230
231     // Array of test patterns to demonstrate different cases
232     String[] testPatterns = {"data", "pattern", "xyz", "a", "data structures", "", "ing"};
233
234
235     // TEST: Pattern appears with TRUE OVERLAPPING OCCURRENCES ("aa" appears 3 times in "aaaa" at positions [0, 1, 2])
236     System.out.println("\n--- Testing TRUE OVERLAPPING Pattern: \"aa\" in Document \"aaaa\" ---");
237     System.out.println("Document: " + overlapDoc.getContent());
238     System.out.println("Pattern: \"aa\" (should find overlapping occurrences)");
239     System.out.println("findFirst result: " + matcher.findFirst(overlapDoc, "aa"));
240     System.out.println("findAll results: " + matcher.findAll(overlapDoc, "aa"));
241     System.out.println("countOccurrences result: " + matcher.countOccurrences(overlapDoc, "aa"));
242     System.out.println("Expected: findAll=[0, 1, 2], count=3");
243
244     // TEST: Pattern appears MULTIPLE TIMES ("data" appears 2 times in doc1)
245     System.out.println("\n--- Testing Pattern: \"data\" in Document 1 ---");
246     System.out.println("Document: " + doc1.getContent());
247     System.out.println("Pattern: \"data\"");
248     System.out.println("findFirst result: " + matcher.findFirst(doc1, "data"));
249     System.out.println("findAll results: " + matcher.findAll(doc1, "data"));
250     System.out.println("countOccurrences result: " + matcher.countOccurrences(doc1, "data"));
251     System.out.println("containsPattern result: " + matcher.containsPattern(doc1, "data"));
252
253     // TEST: Pattern appears MULTIPLE TIMES ("pattern" appears 2 times in doc2)
254     System.out.println("\n--- Testing Pattern: \"pattern\" in Document 2 ---");
255     System.out.println("Document: " + doc2.getContent());
256     System.out.println("Pattern: \"pattern\"");
257     System.out.println("findFirst result: " + matcher.findFirst(doc2, "pattern"));
258     System.out.println("findAll results: " + matcher.findAll(doc2, "pattern"));
259     System.out.println("countOccurrences result: " + matcher.countOccurrences(doc2, "pattern"));
260     System.out.println("containsPattern result: " + matcher.containsPattern(doc2, "pattern"));
261
262     // TEST: Pattern appears MULTIPLE TIMES ("pattern" appears 3 times in doc3)
263     System.out.println("\n--- Testing Pattern: \"pattern\" in Document 3 ---");
264     System.out.println("Document: " + doc3.getContent());

```

```

265     System.out.println("Pattern: \"pattern\"");
266     System.out.println("findFirst result: " + matcher.findFirst(doc3, "pattern"));
267     System.out.println("findAll results: " + matcher.findAll(doc3, "pattern"));
268     System.out.println("countOccurrences result: " + matcher.countOccurrences(doc3, "pattern"));
269     System.out.println("containsPattern result: " + matcher.containsPattern(doc3, "pattern"));
270
271     // TEST: Pattern appears ONCE ("Computer" appears 1 time in doc1)
272     System.out.println("\nPattern: \"Computer\" (appears once)");
273     System.out.println("containsPattern result: " + matcher.containsPattern(doc1, "Computer"));
274     System.out.println("countOccurrences result: " + matcher.countOccurrences(doc1, "Computer"));
275     System.out.println("findFirst result: " + matcher.findFirst(doc1, "Computer"));
276     System.out.println("findAll results: " + matcher.findAll(doc1, "Computer"));
277
278     System.out.println("\n--- Edge Cases Testing ---");
279
280     // TEST: Pattern appears NOT AT ALL ("xyz" does not appear in doc1)
281     System.out.println("\nPattern: \"xyz\" (not found)");
282     System.out.println("containsPattern result: " + matcher.containsPattern(doc1, "xyz"));
283     System.out.println("countOccurrences result: " + matcher.countOccurrences(doc1, "xyz"));
284
285     // TEST: Empty text (edge case - searching in empty document)
286     TextDocument emptyDoc = new TextDocument(4, "Empty", "");
287     System.out.println("\nEmpty text with pattern \"data\" (text is empty)");
288     System.out.println("Document content: \"\"");
289     System.out.println("findFirst result: " + matcher.findFirst(emptyDoc, "data"));
290     System.out.println("countOccurrences result: " + matcher.countOccurrences(emptyDoc, "data"));
291     System.out.println("findAll results: " + matcher.findAll(emptyDoc, "data"));
292
293     // TEST: Empty pattern (edge case)
294     System.out.println("\nPattern: \"\" (empty pattern)");
295     System.out.println("findFirst result: " + matcher.findFirst(doc1, ""));
296     System.out.println("findAll results: " + matcher.findAll(doc1, ""));
297     System.out.println("countOccurrences result: " + matcher.countOccurrences(doc1, ""));
298
299     // TEST: Both empty text and empty pattern (edge case)
300     System.out.println("\nBoth text and pattern are empty");
301     System.out.println("findFirst result: " + matcher.findFirst(emptyDoc, ""));
302     System.out.println("findAll results: " + matcher.findAll(emptyDoc, ""));
303     System.out.println("countOccurrences result: " + matcher.countOccurrences(emptyDoc, ""));
304
305     // TEST: Pattern longer than text (edge case)
306     System.out.println("\nPattern: \"this pattern is very long\" (longer than text)");
307     System.out.println("findFirst result: " + matcher.findFirst(doc1, "this pattern is very long"));
308
309     // TEST: Pattern equal to text (edge case)
310     System.out.println("\nTesting with pattern equal to text");
311     System.out.println("Document: " + doc1.getContent());
312     System.out.println("Pattern: \"data structures and data algorithms\"");
313     System.out.println("findFirst result: " + matcher.findFirst(doc1, "data structures and data algorithms"));
314     System.out.println("findAll results: " + matcher.findAll(doc1, "data structures and data algorithms"));
315     System.out.println("countOccurrences result: " + matcher.countOccurrences(doc1, "data structures and data algorithms"));
316     System.out.println("containsPattern result: " + matcher.containsPattern(doc1, "data structures and data algorithms"));
317 }
318 }

```

This is the Main class, which has been implemented for orchestration, execution verification, and runtime metrics gathering across distinct sorting algorithms and string parsing routines. It contains a static volatile control state field to eliminate Dead Code Elimination side-effects during execution tracking. `main()`, `generateTestData()`, and `demonstratePatternMatching()` serve as the primary execution paths of this controller. `main()` configures test record instances, executes initial verification sequences, runs structured nested performance loops tracking execution benchmarks across randomized or ordered arrays, accumulates side-effect parameters into a runtime checksum validation log, and delegates control to string evaluation blocks. `generateTestData()` instantiates linear data sequences dynamically configured into randomized states, ascending continuous ranges, or inverted descending orders.

demonstratePatternMatching() issues precise string verification routines by pushing pattern metrics—including edge conditions, empty boundary structures, and duplicate character overlap offsets—to validate execution pathways.

PatternMatcher

```

1  import java.util.ArrayList;
2  import java.util.List;
3
4  ✓ public class PatternMatcher {
5
6      // Part E - String Pattern Matching
7  ✓ public int findFirst(String text, String pattern) {
8      if (pattern == null) {
9          return -1;
10     }
11
12     if (pattern.isEmpty()) {
13         return text == null ? -1 : 0;
14     }
15
16     if (text == null || pattern.length() > text.length()) {
17         return -1;
18     }
19
20     for (int startIndex = 0; startIndex <= text.length() - pattern.length(); startIndex++) {
21         if (matchesAt(text, pattern, startIndex)) {
22             return startIndex;
23         }
24     }
25
26     return -1;
27 }
28
29 // Implement a brute-force string pattern-matching algorithm
30 ✓ public List<Integer> findAll(String text, String pattern) {
31     List<Integer> matches = new ArrayList<>();
32
33     if (pattern == null || text == null) {
34         return matches;
35     }
36
37     if (pattern.isEmpty()) {
38         return matches;
39     }
40
41     if (pattern.length() > text.length()) {
42         return matches;
43     }
44
45     for (int startIndex = 0; startIndex <= text.length() - pattern.length(); startIndex++) {
46         if (matchesAt(text, pattern, startIndex)) {
47             matches.add(startIndex);
48         }
49     }
50
51     return matches;
52 }
53
54 // Use a brute-force pattern-matching approach
55 ✓ public int countOccurrences(String text, String pattern) {
56     if (pattern == null || text == null) {
57         return 0;

```

```

58     }
59
60     if (pattern.isEmpty() || pattern.length() > text.length()) {
61         return 0;
62     }
63
64     int occurrenceCount = 0;
65     for (int startIndex = 0; startIndex <= text.length() - pattern.length(); startIndex++) {
66         if (matchesAt(text, pattern, startIndex)) {
67             occurrenceCount++;
68         }
69     }
70
71     return occurrenceCount;
72 }
73
74 // Search for a pattern inside the content of TextDocument objects
75 // findFirst(text, pattern): Return the index of the first occurrence of the pattern, or -1 if the pattern is not found
76 ✓ public int findFirst(TextDocument document, String pattern) {
77     if (document == null) {
78         return -1;
79     }
80
81     return findFirst(document.getContent(), pattern);
82 }
83
84 // findAll(text, pattern): Return or print all starting indices where the pattern occurs.
85 ✓ public List<Integer> findAll(TextDocument document, String pattern) {
86     if (document == null) {
87         return new ArrayList<>();
88     }
89
90     return findAll(document.getContent(), pattern);
91 }
92
93 // countOccurrences(text, pattern): Return the number of times the pattern occurs in the text.
94 ✓ public int countOccurrences(TextDocument document, String pattern) {
95     if (document == null) {
96         return 0;
97     }
98
99     return countOccurrences(document.getContent(), pattern);
100 }
101
102 // containsPattern(text, pattern): Return true if the pattern exists in the text, otherwise false.
103 public boolean containsPattern(String text, String pattern) {
104     return findFirst(text, pattern) != -1;
105 }
106
107 // containsPattern(document, pattern): Return true if the pattern exists in the document, otherwise false.
108 ✓ public boolean containsPattern(TextDocument document, String pattern) {
109     if (document == null) {

```

```

110         return false;
111     }
112
113     return containsPattern(document.getContent(), pattern);
114 }
115
116 // Helper method to check if pattern matches at a specific position
117 ✓ private boolean matchesAt(String text, String pattern, int startIndex) {
118     if (text == null || pattern == null) {
119         return false;
120     }
121
122     if (startIndex < 0 || startIndex + pattern.length() > text.length()) {
123         return false;
124     }
125
126     for (int patternIndex = 0; patternIndex < pattern.length(); patternIndex++) {
127         if (text.charAt(startIndex + patternIndex) != pattern.charAt(patternIndex)) {
128             return false;
129         }
130     }
131
132     return true;
133 }
134 }

```

This is the PatternMatcher class, which has been implemented for raw string segmentation and substring analysis using a brute-force pattern-matching mechanism. It contains methods without explicit instance variables. findFirst(), findAll(), countOccurrences(), containsPattern(), and the overloaded variations accepting TextDocument references operate as the core interfaces of this string utility, alongside the private helper matchesAt(). findFirst() scans the sequence character-by-character to intercept the initial matching boundary index or yields a -1 error flag upon a complete structural miss. findAll() collects every valid offset index into a linear list layout by advancing the window position incrementally across overlapping boundaries. countOccurrences() tracks total sequence frequency metrics by computing matching locations within the search boundaries. containsPattern() evaluates basic truth states by validating the results of localized boundary discovery. matchesAt() isolates positional character extraction routines, running element-by-element comparisons between sequence offsets to confirm structural alignment.

QuickSort

```

1  //Pivot Strategy will be calways hoosing the last element as the pivot
2  ✓ public class QuickSort {
3
4      //public quicksort method that starts the recursion
5  ✓ public void quickSort(StudentRecord[] students) {
6
7      if (students == null || students.length <= 1)
8          return;
9
10     quickSort(students, 0, students.length - 1);
11 }
12
13 //private method that does the brunt of the work and the rest of the recursion part
14 ✓ private void quickSort(StudentRecord[] students, int high, int low){
15
16     //base case
17     if (low < high){
18
19         // Partition the array and get the pivot index
20         int pivotIndex = partition(students, low, high);
21
22         // Sort the left side
23         quickSort(students, low, pivotIndex - 1);
24
25         // Sort the right side
26         quickSort(students, pivotIndex + 1, high);
27     }
28
29 }
30
31
32 ✓ private int partition(StudentRecord[] students, int high, int low){
33
34     // Choose the last element as the pivot
35     StudentRecord pivot = students[high];
36
37     // Index of the smaller element
38     int i = low - 1;
39
40     for (int j = low; j < high; j++) {
41
42         // Compare grades
43         if (students[j].getGrade() <= pivot.getGrade()) {
44
45             i++;
46
47             swap(students, i, j);
48         }
49     }
50
51     // Place the pivot in its correct position
52     swap(students, i + 1, high);
53
54     return i + 1;
55
56 }
57
58 //method to swap 2 values
59 ✓ private void swap(StudentRecord[] students, int i, int j){
60
61     StudentRecord temp = students[i];
62     students[i] = students[j];
63     students[j] = temp;
64 }
65 }

```

This is the QuickSort class, which has been implemented for in-place array partitioning and sorting using a divide-and-conquer recursion mechanism. It contains methods without explicit instance variables. `quickSort()`, `partition()`, and `swap()` are the core methods to operate this sorting sequence. `quickSort()` initiates and manages the subproblem processing boundaries,

splitting the data segments based on structural indices. `partition()` establishes the last element of the target array segment as the structural pivot, conducting single-pass linear scans to segregate lower grade records to the left boundary and higher grade records to the right boundary before inserting the pivot at its correct absolute position. `swap()` alternates the explicit array indices of two discrete elements to execute structural replacements during the partitioning sequence.

SimpleSort

```
1  import java.util.List;
2
3  ✓ public class SimpleSort {
4
5      // Part D - Sorting Algorithm Comparison
6      // Implement or reuse a simple sorting algorithm for comparison.
7      // Sort StudentRecord objects by studentNumber or grade (insertion sort).
8  ✓ public void sortByStudentNumber(StudentRecord[] records) {
9      if (records == null || records.length < 2) {
10         return;
11     }
12
13     for (int currentIndex = 1; currentIndex < records.length; currentIndex++) {
14         StudentRecord currentRecord = records[currentIndex];
15         int insertionIndex = currentIndex;
16
17         while (insertionIndex > 0 && compareByStudentNumber(records[insertionIndex - 1], currentRecord) > 0) {
18             records[insertionIndex] = records[insertionIndex - 1];
19             insertionIndex--;
20         }
21
22         records[insertionIndex] = currentRecord;
23     }
24 }
25
26 ✓ public void sortByGrade(StudentRecord[] records) {
27     if (records == null || records.length < 2) {
28         return;
29     }
30
31     for (int currentIndex = 1; currentIndex < records.length; currentIndex++) {
32         StudentRecord currentRecord = records[currentIndex];
33         int insertionIndex = currentIndex;
34
35         while (insertionIndex > 0 && compareByGrade(records[insertionIndex - 1], currentRecord) > 0) {
36             records[insertionIndex] = records[insertionIndex - 1];
37             insertionIndex--;
38         }
39
40         records[insertionIndex] = currentRecord;
41     }
42 }
43
44 ✓ public void sortByStudentNumber(List<StudentRecord> records) {
45     if (records == null || records.size() < 2) {
46         return;
47     }
48
49     for (int currentIndex = 1; currentIndex < records.size(); currentIndex++) {
50         StudentRecord currentRecord = records.get(currentIndex);
51         int insertionIndex = currentIndex;
52
53         while (insertionIndex > 0 && compareByStudentNumber(records.get(insertionIndex - 1), currentRecord) > 0) {
54             records.set(insertionIndex, records.get(insertionIndex - 1));
55             insertionIndex--;
56         }
57     }
58 }
```

```

57
58         records.set(insertionIndex, currentRecord);
59     }
60 }
61
62 ✓ public void sortByGrade(List<StudentRecord> records) {
63     if (records == null || records.size() < 2) {
64         return;
65     }
66
67     for (int currentIndex = 1; currentIndex < records.size(); currentIndex++) {
68         StudentRecord currentRecord = records.get(currentIndex);
69         int insertionIndex = currentIndex;
70
71         while (insertionIndex > 0 && compareByGrade(records.get(insertionIndex - 1), currentRecord) > 0) {
72             records.set(insertionIndex, records.get(insertionIndex - 1));
73             insertionIndex--;
74         }
75
76         records.set(insertionIndex, currentRecord);
77     }
78 }
79
80 ✓ private int compareByStudentNumber(StudentRecord leftRecord, StudentRecord rightRecord) {
81     if (leftRecord == null || rightRecord == null) {
82         throw new IllegalArgumentException("StudentRecord instances must not be null.");
83     }
84
85     return Integer.compare(leftRecord.getStudentNumber(), rightRecord.getStudentNumber());
86 }
87
88 ✓ private int compareByGrade(StudentRecord leftRecord, StudentRecord rightRecord) {
89     if (leftRecord == null || rightRecord == null) {
90         throw new IllegalArgumentException("StudentRecord instances must not be null.");
91     }
92
93     return Double.compare(leftRecord.getGrade(), rightRecord.getGrade());
94 }
95 }

```

This is the SimpleSort class, which has been implemented for quadratic-time element sorting utilizing an in-place insertion sort mechanism over array and list structures. It contains overloaded variants without explicit instance variables. `sortByStudentNumber()`, `sortByGrade()`, `compareByStudentNumber()`, and `compareByGrade()` serve as the core operations of this utility. The sorting methods process data elements by progressively generating a sorted subsegment at the lower boundary, extracting the record at the active index location, shifting larger preceding entities step-by-step to the right, and plunging the current target record into its correct sequential offset. `compareByStudentNumber()` and `compareByGrade()` isolate property evaluation routines by invoking structural wrappers to compare integer identifier fields and double-precision grade fields, throwing illegal argument signals when encountering uninstantiated records.

StudentRecord

```
1  ✓ public class StudentRecord {
2      private int studentNumber;
3      private String name;
4      private double grade;
5      private int submissionTime; // for example, minutes after deadline or timestamp order
6
7      // constructor
8  ✓ public StudentRecord(int studentNumber, String name, double grade, int submissionTime){
9      this.studentNumber = studentNumber;
10     this.name = name;
11     this.grade = grade;
12     this.submissionTime = submissionTime;
13 }
14
15 //getters
16 public int getStudentNumber(){
17     return studentNumber;
18 }
19
20 public String getName(){
21     return name;
22 }
23
24 public double getGrade(){
25     return grade;
26 }
27
28 public int getSubmissionTime(){
29     return submissionTime;
30 }
31
32 //setters
33 public void setStudentNumber(int studentNumber){
34     this.studentNumber = studentNumber;
35 }
36
37 public void setName(String name){
38     this.name = name;
39 }
40
41 public void setGrade(double grade){
42     this.grade = grade;
43 }
44
45 public void setSubmissionTime(int submissionTime){
46     this.submissionTime = submissionTime;
47 }
48
49 //toString method
50 public String toString(){
51     return studentNumber + " -> " + name + " | " + grade + " | " + submissionTime;
52 }
53
54 }
```

This is the StudentRecord class, which has been implemented for basic data encapsulation and representation of individual academic records. It contains instance variables, an explicit constructor, standard accessors, and mutators. `getStudentNumber()`, `getName()`, `getGrade()`, `getSubmissionTime()`, `setStudentNumber()`, `setName()`, `setGrade()`, `setSubmissionTime()`, and `toString()` serve as the core operations of this data container. The constructor initializes internal state attributes including primitive integer identification keys, string literals for names, double-precision evaluation grades, and relative integer submission timelines. The accessor and mutator methods expose controlled hooks to extract or alter field attributes safely from external

components. `toString()` overrides the default object layout to generate a structured string sequence capturing a snapshot representation of the entity properties.

TextDocument

```
1  ✓ public class TextDocument {
2      private int id;
3      private String title;
4      private String content;
5
6      // constructor
7  ✓ public TextDocument(int id, String title, String content){
8      this.id = id;
9      this.title = title;
10     this.content = content;
11 }
12
13 //getters
14 public int getId(){
15     return id;
16 }
17
18 public String getTitle(){
19     return title;
20 }
21
22 public String getContent(){
23     return content;
24 }
25
26 //setters
27 public void setId(int id){
28     this.id = id;
29 }
30
31 public void setTitle(String title){
32     this.title = title;
33 }
34
35 public void setContent(String content){
36     this.content = content;
37 }
38
39 //toString method
40 public String toString(){
41     return id + "->" + title + "|" + content;
42 }
43 }
```

This is the `TextDocument` class, which has been implemented for data encapsulation and tracking of individual text items. It contains instance variables, an explicit constructor, standard accessors, and mutators. `getId()`, `getTitle()`, `getContent()`, `setId()`, `setTitle()`, `setContent()`, and `toString()` serve as the core operations of this data container. The constructor initializes internal state attributes including unique integer identification values, string descriptions for titles, and textual

body strings for content. The accessor and mutator methods expose controlled hooks to extract or alter field attributes safely from external components. `toString()` overrides the default object layout to generate a single combined string sequence capturing a snapshot representation of the document properties.

MergeSort

```
1  ✓ public class MergeSort {
2
3      //mergeSort public method, this method is the one used in main, while the private one does the brute work
4  ✓  public void mergeSort1(StudentRecord[] students){
5
6          //check if array is empty and if it has only 1 or less elements
7          if (students == null || students.length <= 1){
8              return;
9          }
10
11          mergeSort2(students, 0, students.length - 1);
12      }
13
14      //private mergeSort method to do the actual sorting
15  ✓  private void mergeSort2(StudentRecord[] students, int left, int right){
16
17          //base case: if there is only one element in array
18          if (left >= right){
19              return;
20          }
21
22          // Divide
23          int middle = (left + right) / 2;
24
25          // Conquer
26          mergeSort2(students, left, middle);
27          mergeSort2(students, middle + 1, right);
28
29          // Combine
30          merge(students, left, middle, right);
31      }
32
33      //merge the two sorted halves
35  ✓  public void merge(StudentRecord[] students, int left, int middle, int right){
36
37          //size of each temporary array
38          int leftSize = middle - left + 1;
39          int rightSize = right - middle;
40
41          //temporary arrays
42          StudentRecord[] leftArray = new StudentRecord[leftSize];
43          StudentRecord[] rightArray = new StudentRecord[rightSize];
44
45          // Copy data into temporary arrays
46          for (int i = 0; i < leftSize; i++) {
47              leftArray[i] = students[left + i];
48          }
49
50          for (int j = 0; j < rightSize; j++) {
51              rightArray[j] = students[middle + 1 + j];
52          }
53
54          // Merge the temporary arrays back together
55          int i = 0;
56          int j = 0;
```

```

57         int k = left;
58
59         while (i < leftSize && j < rightSize) {
60
61             // Compare grades
62             if (leftArray[i].getGrade() <= rightArray[j].getGrade()) {
63                 students[k] = leftArray[i];
64                 i++;
65             }
66             else {
67                 students[k] = rightArray[j];
68                 j++;
69             }
70
71             k++;
72         }
73
74         // Copy any remaining elements from left array
75         while (i < leftSize) {
76             students[k] = leftArray[i];
77             i++;
78             k++;
79         }
80
81         // Copy any remaining elements from right array
82         while (j < rightSize) {
83             students[k] = rightArray[j];
84             j++;
85             k++;
86         }
87     }
88 }
89 }

```

This is the MergeSort class, which has been implemented for stable, out-of-place array sorting using a divide-and-conquer recursion mechanism. It contains methods without explicit instance variables. `mergeSort1()`, `mergeSort2()`, and `merge()` are the core methods to operate this sorting logic. `mergeSort1()` acts as the public interface that validates input constraints before delegating to the recursive pipeline. `mergeSort2()` repeatedly bisects the data array segments by calculating midpoint offsets, forcing down into smaller subproblems before invoking the modification step. `merge()` handles the reconstruction sequence by allocating isolated temporary left and right storage structures, populating them with record snapshots, running linear index comparisons against student grades to prioritize sequential assembly, and copying any left-over records back into the original array reference.

Screenshots or copied output from test runs:.

```
[Running] cd "c:\GithubClone\CSI_Lab4\" && javac Main.java && java Main
```

```
=====PART A=====
```

```
Before sorting:
```

```
1001 -> Alice | 82.5 | 5  
1002 -> Bob | 91.0 | 2  
1003 -> Charlie | 75.5 | 7  
1004 -> David | 88.0 | 1  
1005 -> Emma | 69.5 | 4  
1006 -> Fatima | 95.0 | 3  
1007 -> George | 81.0 | 6  
1008 -> Hannah | 78.5 | 8  
1009 -> Ian | 86.5 | 9  
1010 -> Julia | 92.0 | 2
```

```
After sorting using Merge Sort:
```

```
1005 -> Emma | 69.5 | 4  
1003 -> Charlie | 75.5 | 7  
1008 -> Hannah | 78.5 | 8  
1007 -> George | 81.0 | 6  
1001 -> Alice | 82.5 | 5  
1009 -> Ian | 86.5 | 9  
1004 -> David | 88.0 | 1  
1002 -> Bob | 91.0 | 2  
1010 -> Julia | 92.0 | 2  
1006 -> Fatima | 95.0 | 3
```

```
=====PART B=====
```

```
Before sorting:
```

```
1005 -> Emma | 69.5 | 4  
1003 -> Charlie | 75.5 | 7  
1008 -> Hannah | 78.5 | 8  
1007 -> George | 81.0 | 6  
1001 -> Alice | 82.5 | 5  
1009 -> Ian | 86.5 | 9  
1004 -> David | 88.0 | 1  
1002 -> Bob | 91.0 | 2  
1010 -> Julia | 92.0 | 2  
1006 -> Fatima | 95.0 | 3
```

```
After sorting using Quick Sort:
```

```
1005 -> Emma | 69.5 | 4  
1003 -> Charlie | 75.5 | 7  
1008 -> Hannah | 78.5 | 8  
1007 -> George | 81.0 | 6
```

Sorted	100	21500	56200	19200	1700
Sorted	1000	213300	3663700	140900	9000
Reverse Sorted	10	2800	1300	2000	1100
Reverse Sorted	100	13400	28600	10400	38300
Reverse Sorted	1000	127900	4300700	214500	1910400

[Anti-DCE Verification Hash: 46904]

--- Testing Sorting by Different Keys ---

--- Sort by Grade (ascending) ---

Before sorting by grade:

1005 -> Emma | 69.5 | 4
 1001 -> Alice | 82.5 | 5
 1010 -> Julia | 92.0 | 2
 1003 -> Charlie | 75.5 | 7
 1006 -> Fatima | 95.0 | 3

After sorting by grade (using Insertion Sort):

1005 -> Emma | 69.5 | 4
 1003 -> Charlie | 75.5 | 7
 1001 -> Alice | 82.5 | 5
 1010 -> Julia | 92.0 | 2
 1006 -> Fatima | 95.0 | 3

--- Sort by Student Number (ascending) ---

Before sorting by student number:

1005 -> Emma | 69.5 | 4
 1001 -> Alice | 82.5 | 5
 1010 -> Julia | 92.0 | 2
 1003 -> Charlie | 75.5 | 7
 1006 -> Fatima | 95.0 | 3

After sorting by student number (using Insertion Sort):

1001 -> Alice | 82.5 | 5
 1003 -> Charlie | 75.5 | 7
 1005 -> Emma | 69.5 | 4
 1006 -> Fatima | 95.0 | 3
 1010 -> Julia | 92.0 | 2

=====PART E - STRING PATTERN MATCHING=====

--- Testing TRUE OVERLAPPING Pattern: "aa" in Document "aaaa" ---

Document: aaaa bbbb cccc

Pattern: "aa" (should find overlapping occurrences)

findFirst result: 0

findAll results: [0, 1, 2]

countOccurrences result: 3

Expected: findAll=[0, 1, 2], count=3

--- Testing Pattern: "data" in Document 1 ---

Document: Computer science uses data structures and data algorithms

Pattern: "data"

findFirst result: 22

findAll results: [22, 42]

countOccurrences result: 2

containsPattern result: true

--- Testing Pattern: "pattern" in Document 2 ---

Document: Learn Java programming with patterns and design patterns

Pattern: "pattern"

findFirst result: 28

findAll results: [28, 48]

countOccurrences result: 2

containsPattern result: true

--- Testing Pattern: "pattern" in Document 3 ---

Document: pattern matching is a pattern recognition technique for patterns in data

Pattern: "pattern"

findFirst result: 0

findAll results: [0, 22, 56]

countOccurrences result: 3

containsPattern result: true

Pattern: "Computer" (appears once)

containsPattern result: true

countOccurrences result: 1

findFirst result: 0

findAll results: [0]

--- Edge Cases Testing ---

```

Pattern: "xyz" (not found)
containsPattern result: false
countOccurrences result: 0

Empty text with pattern "data" (text is empty)
Document content: ""
findFirst result: -1
countOccurrences result: 0
findAll results: []

Pattern: "" (empty pattern)
findFirst result: 0
findAll results: []
countOccurrences result: 0

Both text and pattern are empty
findFirst result: 0
findAll results: []
countOccurrences result: 0

Pattern: "this pattern is very long" (longer than text)
findFirst result: -1

Testing with pattern equal to text
Document: Computer science uses data structures and data algorithms
Pattern: "data structures and data algorithms"
findFirst result: 22
findAll results: [22]
countOccurrences result: 1
containsPattern result: true

[Done] exited with code=0 in 1.215 seconds

```

Part D - Sorting Algorithm Comparison

=====PART D - SORTING ALGORITHM COMPARISON=====

Comparison Table (Execution Time in nanoseconds):

Dataset Type	Size	Merge Sort	Quick Sort	Heap Sort	Simple Sort
Random	10	9200	4900	7100	38400
Random	100	74500	44000	93300	238500
Random	1000	1049200	681800	422700	5173500
Sorted	10	4900	36600	6700	9500
Sorted	100	43300	66500	33300	3200
Sorted	1000	208200	4132800	320500	15200
Reverse Sorted	10	4200	1800	2700	1500
Reverse Sorted	100	17800	54900	16900	75100
Reverse Sorted	1000	169800	3181600	175200	14243400

[Anti-DCE Verification Hash: 46212]

- Random Case (Size 1000): Heap Sort performed best because it avoids the memory allocation overhead of Merge Sort while strictly guaranteeing $O(n \log n)$ time complexity.
- Sorted Case: Simple Sort performed best because it achieves its best-case time complexity of $O(n)$, requiring only a single linear scan with minimal comparisons.

- Reverse Sorted Case: Merge Sort and Heap Sort performed best because they maintain an optimal $O(n \log n)$ upper bound, preventing the $O(n^2)$ worst-case degradation seen in Simple Sort and Quick Sort.

Part F - Java Collections and Library Comparison

	Custom merge-sort	Collections.sort or Arrays.sort
Correctness	Correctly sorts elements based on grade, but relies heavily on student-managed boundary conditions.	100% correct. Because it is verified and tested by the industry.
Efficiency	Strict $O(n \log n)$ time complexity across all cases, but it suffers from high constant factor overhead on small subarrays.	Can run in $O(n \log n)$ average or worst-case time but optimizes to $O(n)$ linear time for partially sorted data.
Ease of Use	Low. It requires manually coding the recursive split, implementing the merge method, and handling index pointer logic.	High. It is achievable via a single API call, passing the array or list directly to the library method.
In-place	Not in-place. It requires allocating an external temporary array of size $O(n)$ during the merge process.	Not in-place. It allocates temporary workspace memory variables to execute the sorting steps.

	Custom quick-sort	Arrays.sort behavior discussion
Efficiency	Custom quick-sort runs in $O(n \log n)$ average time but degrades to quadratic $O(n^2)$ time if a static pivot encounters sorted or reverse-sorted data.	The library utilizes an optimized Dual-Pivot strategy to eliminate worst-case degradation.
Stability	Unstable. Long-distance element swaps across partitioning barriers routinely alter the original relative placement of equal keys.	Unstable. Long-distance element swaps across partitioning barriers routinely alter the original relative placement of equal keys.

In-place	It manipulates elements directly inside the input array boundaries, avoiding explicit secondary array duplication.	It manipulates elements directly inside the input array boundaries, avoiding explicit secondary array duplication.
Practical Use	Custom single-pivot systems are vulnerable to call-stack errors on specific configurations.	The library variant is an industry standard optimized directly for CPU cache locality and high-throughput production environments.

	Custom heap-sort	PriorityQueue-based sorting discussion
Correctness	It leverages binary heap properties to extract maximum or minimum elements sequentially, resulting in an accurately sorted collection.	It leverages binary heap properties to extract maximum or minimum elements sequentially, resulting in an accurately sorted collection.
Stability	Unstable; structural swaps within the tree layout routinely scramble the relative index layout of identical keys.	Unstable; structural swaps within the tree layout routinely scramble the relative index layout of identical keys.
In-place	It operates directly within the input array boundaries, maintaining a strict $O(1)$ auxiliary space footprint via iterative sift-down logic.	It requires allocating an entirely new object instance in heap memory, generating an auxiliary space footprint of $O(n)$.
Practical Use	It is optimal for embedded or resource-constrained software systems where strict, predictable memory ceilings are non-negotiable.	The processing is intended for continuous data streaming apps or Top-K retrieval elements rather than batch-sorting existing array structures

	Custom simple sort	Collections.sort or Arrays.sort
Correctness	It successfully organizes input data arrays into verified sorted sequences.	It successfully organizes input data arrays into verified sorted sequences.
Efficiency	It can operates at a average and worst-case quadratic $O(n^2)$ complexity curve, making it unusable for large data blocks.	It scales at an optimal $O(n \log n)$ runtime limit, keeping processing speed high for substantial production datasets.

Stability	Stable. It preserves original sequence alignments for equal data elements when custom loops enforce sequential insertion limits correctly.	Guaranteed stable. TimSort/MergeSort strictly preserves the original relative order of equal elements naturally.
Practical Use	It serves as an educational framework to highlight localized loop logic, though it is used internally by hybrid engines for micro-arrays.	It operates as a general-purpose production interface capable of managing industrial-scale object arrays.

	Custom pattern matcher	String.indexOf, String.contains, or regular expressions
Correctness	It accurately resolves structural matching, returning matching initial pointers or outputting -1 if the string cannot be found.	It accurately resolves structural matching, returning matching initial pointers or outputting -1 if the string cannot be found.
Efficiency	Custom brute-force executes in $O(n*m)$ time, checking character by character sequentially.	Java's String.indexOf can evaluate blocks of bytes simultaneously, bypassing standard single-loop boundaries.
In-place	It operates directly across existing memory segments without allocating massive duplicate text buffers during the comparison loop.	It operates directly across existing memory segments without allocating massive duplicate text buffers during the comparison loop.
Practical Use	It is primarily used for academic validation to demonstrate basic pointer scanning; its lack of performance scaling makes it unsafe for processing unstructured production data streams.	It serves as a core production utility, thoroughly optimized to handle massive text streams, memory safety constraints, and variable file inputs natively across real-world software applications.

Complexity analysis table:

Operation	Best-case	Average-case	Worst-case	Space complexity	Stable?	In-Place?
-----------	-----------	--------------	------------	------------------	---------	-----------

Merge-Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes, because during the merge step, if two objects have equal keys, the element from the left subarray is placed first, keeping the original order	No, because it requires additional temporary arrays to merge the two sorted halves.
Quick-sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No, because swapping elements during partitioning can change the original order of objects with equal keys.	Yes, because it rearranges elements directly inside the original array and only uses a small amount of extra memory for recursion.
Heap-sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No, A stable sorting algorithm keeps the original order of elements with equal keys, and heap sort doesn't	Yes, because we sort directly inside the original array.
Insertion sort or	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes, because	Yes, because it

selection sort					equal elements are not swapped past one another, so their original order is preserved.	rearranges elements within the original array without requiring an additional array.
Brute-force pattern matching (n = length of text, m = length of pattern)	$O(n)$	$O(nm)$	$O(nm)$	$O(1)$	N/A	N/A

Answers to the required written questions:

1. Explain why merge-sort is considered a divide-and-conquer algorithm.
 - Merge-Sort is considered a divide-and-conquer algorithm because it takes the data and splits it in half, then takes the right and left halves then splits it again. This is done recursively until both halves are sorted and then combines them again.
2. Explain why merge-sort has $O(n \log n)$ running time.
 - Merge-sort has an $O(n \log n)$ running time because it divides the input array into two smaller halves repeatedly until each subarray contains one element. This dividing process creates $\log(n)$ levels of recursion. At each level, the merge step compares and combines all elements, which takes $O(n)$ time. Since there are $\log(n)$ levels and each level requires $O(n)$ work, the total running time is $O(n \log n)$.
3. Explain how quick-sort partitions the input.
 - Quick-sort partitions the input by selecting a pivot element and rearranging the array so that elements smaller than the pivot are placed on one side and elements larger than the pivot are placed on the other side. After partitioning, the pivot is in its correct sorted position. The algorithm then recursively applies the same process to the smaller subarrays on the left and right of the pivot until the entire array is sorted.
4. Explain how pivot selection can affect quick-sort performance.

- The choice of pivot can greatly affect the performance of quick-sort. If the pivot divides the array into two roughly equal parts, quick-sort performs efficiently with a time complexity of $O(n \log n)$. However, if the pivot is consistently the smallest or largest element, the partitions become unbalanced, causing quick-sort to take $O(n^2)$ time in the worst case. Choosing a good pivot helps maintain better performance.
5. Explain why heap-sort has $O(n \log n)$ running time.
- Heap-sort has an $O(n \log n)$ running time because it first builds a heap from the input data, which takes $O(n)$ time. After the heap is created, the maximum or minimum element is repeatedly removed, and the heap is adjusted using heapify. Each removal requires $O(\log n)$ time, and this operation is performed n times. Therefore, the total running time is $O(n \log n)$.
6. Compare merge-sort, quick-sort, and heap-sort in terms of stability and memory usage.
- Stability:
 - o Merge-sort: It is a stable sort. The elements in the array are divided into subarrays. The merge-sort process starts from the bottom and compares the left element and the right element. If both elements in the array are the same, the sort process would keep the left element on the left and the right element on the right. The relative order doesn't change.
 - o Quick-sort: It is not a stable sort. The relative order of the element, which contain the same key may change due to the position switch between pointers. The reason is that the switches are triggered by comparing the keys to the pivot. If both pointers contain the same key and are less than the pivot, these two elements would switch positions. This process may cause an unstable sort.
 - o Heap-sort: It is not a stable sort. The relative order of the duplicated array may change through the swapping between parent nodes and child nodes during the rebuild process in heapify.
 - Memory usage:
 - o Merge-sort: This sort takes $O(1)$ auxiliary space because this sorting method divides the parent array by manipulating index boundaries until the base case is reached, which is the case when the subarray length is less than 2. The temporary array memory is used during the merge process to combine the sorted subarrays back into the original array.
 - o Quick-sort: This sort usually takes $O(\log n)$ auxiliary space but $O(n)$ in the worst case. The memory usage depends on how the developers chose the pivot. To reduces the memory usage, the median element is a common choice for the pivot. In this case the height of the recursion tree would be the least. The call stack would only contain $\log n$ layers of stack frames. However, if the pivot the first or

the last element in the array, and the tree is shaped as a line, the height of the tree would be n and takes $O(n)$ auxiliary space to run the algorithm.

- Heap-sort: This sort takes $O(1)$ auxiliary space because this sorting method operates directly within the input array, making it strictly an in-place sorting algorithm.
7. Explain why simple sorting algorithms may still be useful for small inputs.
 - Simple sorting algorithms are helpful for operating small size input. There is no extra overhead created during the operation. Even though the theoretical running time of the simple sorting algorithms is larger than the running time of other sorting algorithms, like merge-sort, quick-sort, and heap-sort, the actual running time of a simple sorting algorithm can be faster.
 8. Explain how brute-force pattern matching works.
 - The brute-force pattern-matching algorithm aligns a pattern string of length m against the starting position of a text string of length n . It scans from left to right, comparing characters one by one between the text and the pattern. If a mismatch occurs during this loop, the algorithm stops checking the current text window immediately, shifts the pattern forward by exactly one position relative to the text, and restarts the check from the first character of the pattern. This shift-and-compare process repeats until either a full match is found or the text index passes the boundary $n - m$, where the pattern can no longer structurally fit.
 9. Explain the worst-case time complexity of brute-force pattern matching.
 - The worst-case time complexity is $O(n*m)$; n is the length of the text string being searched, and m is the length of the pattern string.. It occurs when the text keeps mismatching until the last successful comparison.
 10. Compare `String.indexOf` with a custom brute-force pattern-matching implementation.
 - Efficiency
 - `String.indexOf`: It executes significantly faster because it utilizes specialized lower-level runtime optimizations to process data blocks simultaneously.
 - Custom brute-force pattern-matching: It executes slower because it sequentially evaluates each character one by one, leading to high processing times on large files.
 - Flexibility
 - `String.indexOf`: It provides limited functionality, designed strictly to locate the single index of the first occurrence. External loops must be added to track multiple matches.
 - Custom brute-force pattern-matching: It offers higher adaptability, allowing developers to integrate multi-match extraction or cumulative frequency counts directly into the internal logic.

Short discussion comparing priority queues, heaps, maps, hash tables, BSTs, balanced search tree concepts, and Java Collections Framework classes:

Data structures manage tradeoffs between key-based retrieval efficiency and element ordering constraints. Priority queues function as abstract data types that optimize for the retrieval of the highest or lowest priority element, which are most commonly implemented using heaps because an array-based binary heap provides $O(1)$ access to the extremum while maintaining $O(\log n)$ insertion and removal times. In contrast, maps represent an abstract framework for managing key-value pairs, which are implemented either via hash tables or binary search trees (BSTs). Hash tables, materialized in the Java Collections Framework as `HashMap`, leverage hash functions to achieve $O(1)$ average-time complexity for insertions and lookups, though they completely sacrifice element ordering and can degrade under severe hash collisions. Standard BSTs offer ordered traversals but can degenerate into linear structures under sequential inputs, necessitating balanced search tree concepts like AVL trees or Red-Black trees to enforce structural balance and guarantee strict $O(\log n)$ performance boundaries. The Java Collections Framework integrates these balanced tree mechanics natively within classes like `TreeMap` and `TreeSet`, offering deterministic ordered iterations at the cost of the raw, constant-time speed provided by hash-based implementations.

Problems encountered and how they were solved:

- During the implementation of the sorting algorithms, a few challenges were encountered. One difficulty was understanding the recursive steps in Merge Sort and Quick Sort, especially how the arrays were divided and combined or partitioned. This was solved by tracing the algorithms with smaller examples and testing each step. Another issue was debugging errors in the helper methods, such as incorrect index calculations in Merge Sort and understanding the partition process in Quick Sort. For Heap Sort, understanding how the heap structure works using an array representation was challenging at first. These issues were solved by reviewing the algorithm steps, testing different cases, and fixing errors through debugging.